CODSOFT Python Programming Internship

Task 5: Contact Book Application

- Submitted By:

  Gourav Singh Rajpoot

- Internship Domain:

  Python Programming

Project

****Contact Book Application Using Python****

 # Objective

The goal of this project is to create a Contact Book Application using Python that helps users manage their contact information efficiently. This application lets users add, view, search, update and delete contacts while keeping track of details like name, phone number, email and address.

 # Problem Statement

When you have a lot of contacts it can be hard to keep track of them all. A Contact Book Application makes it easy to store and manage contact information.

The application should be able to:

* Add contacts

* Show saved contacts

* Search for contacts

* Update contact details

* Delete contacts

* Check if phone numbers are valid

* Make sure there are no phone numbers

Tools and Technologies Used

Tool	Purpose
Python 3	Programming Language
VS Code / IDLE	Code Editor
Command Prompt / Terminal	Program Execution

System Requirements

- **Hardware Requirements**
  * Computer/Laptop
  * Minimum 2 GB RAM

- **Software Requirements**
  * Python 3.x
  * Any Code Editor (VS Code, PyCharm, IDLE)

Algorithm

- **Step 1**

Create a list called `contacts` that's empty.

- **Step 2**

Show the Contact Book menu to the user.

- **Step 3**

Let users add contact information, including:

* Name

* Phone Number

* Email

* Address

- **Step 4**

Check if the phone number is valid by making sure it has 10 digits.

- **Step 5**

Before saving a contact check if the phone number is already in use.

- **Step 6**

Give users options to:

* View Contacts

* Search Contacts

* Update Contacts

* Delete Contacts

- **Step 7**

Show the results based on what the user chose to do.

- **Step 8**

Keep showing the menu until the user chooses to Exit.

 # Source Code

****Contact Book Application Using Python****

```
# Contact Book Application

contacts = []

def is_valid_phone(phone):
    return phone.isdigit() and len(phone) == 10

def display_contact(contact):
    print("Name   :", contact["Name"])
    print("Phone  :", contact["Phone"])
    print("Email  :", contact["Email"])
    print("Address:", contact["Address"])

while True:
    print("\n===== CONTACT BOOK =====")
    print("1. Add Contact")
    print("2. View Contacts")
    print("3. Search Contact")
    print("4. Update Contact")
    print("5. Delete Contact")
```

```
print("6. Exit")

choice = input("Enter your choice (1-6): ")

if choice == "1":
    name = input("Enter Name: ")
    phone = input("Enter Phone Number: ")

    if not is_valid_phone(phone):
        print("Invalid phone number! Please enter 10 digits only.")
        continue

    duplicate = False
    for contact in contacts:
        if contact["Phone"] == phone:
            duplicate = True
            break

    if duplicate:
        print("This phone number already exists.")
        continue

    email = input("Enter Email: ")
    address = input("Enter Address: ")

    contact = {
        "Name": name,
        "Phone": phone,
```

```
        "Email": email,
        "Address": address
    }

    contacts.append(contact)
    print("Contact Added Successfully!")

elif choice == "2":
    if not contacts:
        print("No Contacts Available.")
    else:
        print("\n===== CONTACT LIST =====")
        for index, contact in enumerate(contacts, start=1):
            print(f"\nContact {index}")
            display_contact(contact)

elif choice == "3":
    search_value = input("Enter name or phone number to search: ").lower()

    found = False

    for contact in contacts:
        if contact["Name"].lower() == search_value or contact["Phone"] == search_value:
            print("\nContact Found")
            display_contact(contact)
            found = True

    if not found:
```

```
        print("Contact Not Found.")

elif choice == "4":
    search_value = input("Enter name or phone number to update: ").lower()

    found = False

    for contact in contacts:
        if contact["Name"].lower() == search_value or contact["Phone"] == search_value:
            print("\nCurrent Contact Details:")
            display_contact(contact)

            new_name = input("Enter New Name: ")
            new_phone = input("Enter New Phone Number: ")

            if not is_valid_phone(new_phone):
                print("Invalid phone number! Please enter 10 digits only.")
                found = True
                break

            duplicate = False
            for other_contact in contacts:
                if other_contact != contact and other_contact["Phone"] == new_phone:
                    duplicate = True
                    break

            if duplicate:
                print("This phone number already exists.")
```

```
            found = True
            break

        new_email = input("Enter New Email: ")
        new_address = input("Enter New Address: ")

        contact["Name"] = new_name
        contact["Phone"] = new_phone
        contact["Email"] = new_email
        contact["Address"] = new_address

        print("Contact Updated Successfully!")
        found = True
        break

  if not found:
    print("Contact Not Found.")

elif choice == "5":
  if not contacts:
    print("No Contacts Available.")
  else:
    print("\n===== CONTACT LIST =====")
    for index, contact in enumerate(contacts, start=1):
      print(f"{index}. {contact['Name']} - {contact['Phone']}")

    try:
      delete_index = int(input("Enter contact number to delete: "))
```

```
            if 1 <= delete_index <= len(contacts):
                removed_contact = contacts.pop(delete_index - 1)
                print(f"{removed_contact['Name']} has been deleted successfully.")
            else:
                print("Invalid contact number!")

        except ValueError:
            print("Please enter a valid number.")

    elif choice == "6":
        print("Exiting Contact Book...")
        break

    else:
        print("Invalid Choice! Please select between 1 and 6.")
```

- **Explanation of the Code**

 # Contact Storage

We use a list called `contacts` to store all the contact information. The Contact Book Application uses this list to keep track of all the contacts.

 # Phone Number Validation

The function `is_valid_phone()` checks if a phone number is valid by making sure it has 10 digits. This is a feature of the Contact Book Application because it helps keep the contact information accurate.

 # Add Contact

The Contact Book Application lets users add contact details, including name, phone number, email and address. This is a feature of the application because it allows users to store new contacts.

 # Duplicate Contact Prevention

Before adding a contact the Contact Book Application checks if the phone number is already in use. This prevents contacts from being added which helps keep the contact information organized.

 # Search Contact

Users can search for contacts using their name or phone number. This is a feature of the Contact Book Application because it makes it easy to find specific contacts.

 # Update Contact

The Contact Book Application lets users update contact details, including name, phone number, email and address. This is a feature because it allows users to keep their contact information up to date.

 # Delete Contact

Users can delete a contact using its contact number/index. This is a feature of the Contact Book Application because it allows users to remove contacts that are no longer needed.

 # View Contacts

The Contact Book Application displays all saved contacts with their details. This is a feature of the application because it allows users to see all their contacts in one place.

- **Sample Output**

===== CONTACT BOOK =====

1. Add Contact

2. View Contacts

3. Search Contact

4. Update Contact

5. Delete Contact

6. Exit

Enter your choice (1-6): 1

Enter Name: xyz

Enter Phone Number: 9876543210

Enter Email: xyz@gmail.com

Enter Address: Kolkata

Contact Added Successfully!

 # Search Example

Enter name or phone number to search: xyz

Contact Found

Name : xyz

Phone : 9876543210

Email : xyz@gmail.com

Address: kolkata

 # Features

* Add contacts to the Contact Book Application
* View all contacts in the Contact Book Application
* Search for contacts in the Contact Book Application by name or phone number
* Update contact information in the Contact Book Application
* Delete contacts from the Contact Book Application
* Validate phone numbers in the Contact Book Application
* Prevent duplicate phone numbers in the Contact Book Application

* Use a menu-driven interface in the Contact Book Application

* Enjoy a user- design in the Contact Book Application

Advantages

* The Contact Book Application is easy to use

* The Contact Book Application Organizes contact information efficiently

* The Contact Book Application Prevents duplicate contact entries

* The Contact Book Application validates phone numbers

* The Contact Book Application supports contact search

* The Contact Book Application is a Beginner-friendly Python project

Learning Outcomes

Through this project we learned about:

* Python basics, which are essential for creating the Contact Book Application

* Lists and dictionaries which are used to store contact information in the Contact Book Application

* Functions, which are used to perform tasks in the Contact Book Application

* Loops, which are used to repeat tasks in the Contact Book Application

* Conditional statements, which are used to make decisions in the Contact Book Application

* User input handling, which is used to get input from users in the Contact Book Application

* Data validation, which is used to check if data is valid in the Contact Book Application

* Search operations, which are used to find contacts in the Contact Book Application

* CRUD operations, which are used to create, read, update and delete contacts in the Contact Book Application

Future Enhancements

* Save contacts to a file, which would allow users to save their contacts permanently

* Import and export contacts, which would allow users to transfer contacts between devices

* Create a graphical user interface, which would make the Contact Book Application more user-friendly

* Add contact categories, which would allow users to organize their contacts

* Add a favourite contacts feature, which would allow users to mark contacts

* Add password protection, which would keep the Contact Book Application secure

#Conclusion

We successfully created the Contact Book Application using Python. The Contact Book Application provides a way to manage contacts through features like adding, viewing, searching, updating and deleting contacts. The Contact Book Application also validates phone numbers. Prevents duplicates making it a practical contact management solution.

Result

We successfully. Tested the Contact Book Application. Users can manage their contact information efficiently through a menu-driven interface. The Contact Book Application supports contact storage, search, update, deletion and validation features making it a useful tool, for anyone who needs to manage contacts.